

PhishDef: URL Names Say It All

Anh Le, Athina Markopoulou
University of California, Irvine
{anh.le, athina}@uci.edu

Michalis Faloutsos
University of California, Riverside
michalis@cs.ucr.edu

Abstract—Phishing is an increasingly sophisticated method to steal personal user information using sites that pretend to be legitimate. In this paper, we take the following steps to identify phishing URLs. First, we carefully select lexical features of the URLs that are resistant to obfuscation techniques used by attackers. Second, we evaluate the classification accuracy when using only lexical features, both automatically and hand-selected, vs. when using additional features. We show that lexical features are sufficient for all practical purposes. Third, we thoroughly compare several classification algorithms, and we propose to use an online method (AROW) that is able to overcome noisy training data. Based on the insights gained from our analysis, we propose PhishDef, a phishing detection system that uses only URL names and combines the above three elements. PhishDef is a highly accurate method (when compared to state-of-the-art approaches over real datasets), lightweight (thus appropriate for online and client-side deployment), proactive (based on online classification rather than blacklists), and resilient to training data inaccuracies (thus enabling the use of large noisy training data).

I. INTRODUCTION

Phishing is continuously evolving and becoming an increasingly sophisticated criminal tool to steal sensitive information and commit crimes on the Internet. According to the latest report from the Anti-Phishing Working Group [7], the number of commercial brands being attacked by phishing just hit a new record: 356 brands in October 2009. With major industry targets, such as, financial and payment services, phishing has caused billions of dollars loss annually [15]. Because of the severity of the problem, the Internet community has put a significant amount of effort into defense mechanisms.

Currently, two of the most popular services that protect the Internet users from visiting phishing sites are the Google Safe Browsing service [1] and the Microsoft Smart Screen service [3]. Both services provide client browsers with URL blacklists. The browsers, in turn, protect users from visiting the blacklisted URLs. The major problem of this protection model is that it is reactive: a phishing URL can only be included in the blacklist if it has already appeared somewhere else, *e.g.*, in a spam email, or has been reported by a user. A proactive model, which can accurately identify new phishing URLs, is highly desirable to better protect the users.

We argue that in order to provide proactive protection, the machine learning classification engine, which is typically used to maintain the blacklists at the server side, must be pushed to the client browser.¹ This would allow new URLs to be classified on-the-fly, at the time the users click on or type in the

URLs. One of the biggest challenges of classifying URLs on-the-fly, as opposed to off-line at the server side, is the latency constraint. The longer it takes to obtain the classification result of a URL, the longer a user has to wait to load that URL, and the worse the user experience. Furthermore, since the page loading time is a decisive factor when benchmarking web browsers, classifying URLs should not introduce high latency.

There are two types of features that can be used in URL classification: *lexical features*, *i.e.*, features readily available from the URL names; and *external features*, *i.e.*, features acquired from queries to remote servers. We refer to lexical and external features together as *full features*. Lexical features are based only on the URL names and are appropriate for implementation at the client. External features rely on the availability of remote servers, introduce additional latency due to the required queries, and consume more resources of the client, *e.g.*, battery life and bandwidth of mobile phones. Nonetheless, one would expect that relying on a more comprehensive set of features would lead to higher classification accuracy. In this paper, we address the following question:

How well can one detect phishing URLs using only lexical features compared to using full features?

To the best of our knowledge, this work is the first to extensively study this question. We show that lexical features are sufficient (*i.e.*, if properly used, they can achieve accuracy comparable to full features), and we propose a system called PhishDef that achieves this goal.

In particular, we first introduce a way to extract lexical features that are resistant to obfuscation. We then thoroughly evaluate the classification accuracy achieved when using lexical features vs. full features with several state-of-the-art learning algorithms on real datasets. More specifically, we consider the following algorithms: batch-based Support Vector Machine (SVM), Online Perceptron (OP), Confidence-Weighted (CW), and Adaptive Regularization of Weights (AROW). We find that, using lexical features results in a modest decrease (about 1%) in classification accuracy compared to using full features; however, the overall accuracy is still high (96–98%). This suggests that using lexical features is sufficient and provides a better latency-accuracy trade-off. Moreover, our proposed obfuscation-resistant lexical features help to boost the overall classification accuracy across all the datasets. In particular, the reduction of error rate is up to 27%. We also observe that state-of-the-art online linear classification algorithms, namely, AROW and CW, are more accurate while imposing less memory and computing overhead compared to other techniques. Furthermore, when there is noise in the training data (noisy labels), AROW outperforms CW. Robustness in a noisy environment is very important because (i) it allows

This work has been supported by awards NSF 0831530 and ARL W911NF-09-2-0053.

¹A couple of months after we developed PhishDef, Google started implementing client-side classification for the Chrome browser; however, Firefox still uses blacklist-based Google Safe Browsing.

for training more comprehensive classification models by working with larger datasets, which typically include noise, such as, blacklisted URLs from Google used in [18]; and (ii) it improves the system’s resilience to poisoning attacks, where attackers attempt to maliciously influence the classification models by injecting mis-labeled data.

Based on the insights gained from our analysis, we propose *PhishDef*, a classification engine that operates at the client side, uses only lexical features, and implements the AROW algorithm. *PhishDef* has the following desired properties:

- **High accuracy:** It has 96–97% classification accuracy, only 1% less than full features.
- **Light-weight:** It has low latency and imposes a modest amount of memory and computation overhead.
- **Proactive approach:** It can classify new URLs on-the-fly, *i.e.*, at the time the user clicks on or enters the URL at the client side, as opposed to reactively relying on blacklists.
- **Resilience to noise:** It maintains high accuracy even when trained with mislabeled data: 95%–86% accuracy when there is 5%–45% noise.

II. BACKGROUND

In this work, we adopt a broad definition of phishing from Whittaker *et al.* [18], which defines a phishing page as “any web page that, without permission, alleges to act on behalf of a third party with the intention of confusing viewers into performing an action with which the viewer would only trust a true agent of the third party.”

In [11], Garera *et al.* studied the structure of phishing URLs and found four distinct categories of obfuscation techniques that phishing URLs use. In this work, we propose features directly extractable from URL strings that address these four common obfuscation techniques.

In [13], Ma *et al.* compared several batch-based learning algorithms for classifying malicious URLs and showed that the combination of host-based and lexical features results in the highest classification accuracy. It also hinted towards the use of lexical features but did not pursue this direction. We extensively evaluate how both batch-based and online algorithms perform when using only lexical instead of full features. In follow-up work [14], Ma *et al.* compared the performance of batch-based algorithms to online algorithms when using full features and found that online algorithms, especially Confidence-Weighted (CW), outperform batch-based algorithms. Our main difference from [14] is that we focus on lexical features instead of full features. In addition, we introduce AROW, which performs as well as CW but outperforms CW when there is noise. To the best of our knowledge, AROW has not been used before in the phishing context.

III. DATASETS AND FEATURE EXTRACTION

A. Malicious and Legitimate URLs

PhishTank. PhishTank [4] is a community site where anyone can submit, verify, and share phishing URLs. We collect our set of phishing URLs during the one month period of June 2010. The set consists of 4,082 verified phishing URLs ordered by their submission time.

MalwarePatrol. MalwarePatrol [2] is a free and user contributed system where anyone can submit suspicious URLs

that may carry malware, viruses, or trojans. We collect 2,001 malicious URLs during the last two weeks of June 2010. We order these URLs by their appearance time. We note that the URLs here are crafted to spread malware while the URLs from PhishTank are crafted to steal sensitive information.

Yahoo Directory. Yahoo provides a generator URL [6], which randomly generates a URL in its directory whenever someone visits it. We used this generator URL in mid June 2010 to collect 4143 random URLs.

Open Directory. DMOZ is one of the largest open directory of the Web maintained by volunteer editors. We collect 4012 random URLs from DMOZ directory in mid June 2010.

For the benign URLs, we order them by the order in which we obtain them. We also note that our methodology of collecting URL datasets is similar to recent work [13], [16].

B. External Feature Collection

We refer to features that require queries to remote servers as external features. For each URL, we acquire external features by querying two different remote servers:

WHOIS. We query the WHOIS server responsible for the top level domain of the URL for its registration information, which includes the primary domain name, the registrar, the registrant, and the registration date.

Team Cymru. We also query Team Cymru server [17] to obtain the network information and the geolocation of each URL. In particular, we obtain the network BGP prefix, the AS number, and the country code.

We note that collecting these external features incur significant latency. On average, the time it takes to collect all external features of an URL in the PhishTank dataset is 1.64 second. The latency depends on a variety of elements, such as, the load of the WHOIS and Team Cymru servers, as well as the geolocations of the WHOIS servers.

C. Feature Extraction

1) *Lexical Features:* Recall that lexical features can be directly extracted from the URL string. We adopt the approach by Ma *et al.* [13], [14] to automatically select binary lexical features. In addition, motivated by the work by Garera *et al.* [11], we propose a number of obfuscation-resistant lexical features. We show through empirical results that these features complement the former set of features and help to capture additional obfuscated phishing URLs.

Automatically Selected Features. The URL string is broken down into multiple tokens. Each token constitutes a binary feature. The delimiters to obtain the tokens are ‘/’, ‘?’, ‘.’, ‘=’, ‘_’, ‘&’, and ‘-’.

Hand-Selected (Obfuscation-Resistant) Features. In [11], Garera *et al.* describe four different URL obfuscation techniques that are commonly used by the attackers: (I) Obfuscating the host with an IP address, (II) Obfuscating the host with another domain, (III) Obfuscating with large host names, and (IV) Domain unknown or misspelled. Here we propose the following hand-selected lexical features to detect the identified obfuscation techniques:

TABLE I
LEXICAL FEATURES OF A PHISHING URL

URL	www.naturenilai.com/form2/paypal/webscr.php?cmd=_login	
Auto-Selected	name=www, name=naturenilai, tld=com, dir=form2, dir=paypal file=webscr, ext=php, arg=cmd, arg=login	
Obfuscation-Resistant	URL	len=54, n_dot=3, blacklist=1 (webscr)
	Domain Name	len=19, IP=0, port=0, n_token=3, n_hyphen=0, max_len=11
	Directory	len=14, n_subdir=2, max_len=6, max_dot=0, max_delim=0
	File Name	len=10, n_dot=1, n_delim=0
	Argument	len=11, n_var=1, max_len=6, max_delim=1

(i) *Features related to the full URL.* These features include the length of the URL, the number of dots in the URL, and whether a blacklisted word appears in the URL. The blacklist we use is similar to the one in [11]. The first two features address Type II obfuscation while the blacklisted words enhance the detection of Type IV obfuscation.

(ii) *Features related to the domain name.* These features include the length of the domain name, whether an IP address or a port number is used in the domain name, the number of tokens of the domain name, the number of hyphens used in the domain name, and the length of the longest token. These features address obfuscation Type I, Type III, and a technique related to Type III, where hyphens are used instead of dots.

(iii) *Features related to the directory.* These mainly address Type II obfuscation and include the length of the directory, the number of sub-directory tokens, the length of the longest sub-directory token, the maximum number of dots and other delimiters used in a sub-directory token.

(iv) *Features related to the file name (page name).* These features include the length of the file name and the number of dots and other delimiters ('_' and '-') used in the file name. These features also address Type II obfuscation, but in this case, the obfuscated host name is put in the file name.

(v) *Features related to the argument part.* URLs that serve pages written in server side scripting languages, such as, php and asp, often have arguments. The features in this category include the length of the argument part, the number of variables, the length of the longest variable value, and the maximum number of delimiters used in a value.

Table I illustrates how we obtain all lexical features.

2) *External Features:* We extract a number of binary features and one real value feature from the responses we receive from the WHOIS and Team Cymru servers. The registration date gives the real value feature indicating the number of days the site has been up. The other pieces of information that we described in Section III-B give the binary features.

Finally, for all the real value features, we shift and scale them so that their values lie between 0 and 1.

D. Summary of Datasets

We prepare the data for the classification algorithms by combining the legitimate with the malicious URL datasets. In total, we have 5 pairs: Yahoo and PhishTank (Yahoo-Phish); Yahoo and MalwarePatrol (Yahoo-Malware); Open Directory and PhishTank (DMOZ-Phish); Open Directory and MalwarePatrol (DMOZ-Malware); and all good and all bad URLs (All Good - All Bad), where we combine Yahoo with Open Directory and PhishTank with MalwarePatrol.

IV. CLASSIFICATION ALGORITHMS

Notation. Denote the features of an URL as a vector $\mathbf{x}$ and its label as $y \in \{1, -1\}$, where 1 indicates the URL is malicious and -1 indicates otherwise. A classification algorithm receives a number of data vectors, $\mathbf{x}_i$, together with their labels, y_i , and trains its model based on these labeled data. Then, given a new data vector, $\mathbf{x}$, the goal of the algorithm is to predict the label, y , of this new data based on its trained model. For all algorithms, the prediction, $h(\mathbf{x})$, is the sign of the inner product between $\mathbf{w}$ and $\mathbf{x}$:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x}) \quad (1)$$

Batched-based vs. Online. A batch-based algorithm initially trains its model based on a batch of labeled data. It then uses the trained model to predict a number of new data. After some time, it retrains its model based on a new batch of labeled data. Meanwhile, an online classification algorithm continuously retrains its model upon receiving each labeled data and makes prediction of a new data using the latest updated model. Because training a model of a batch-based algorithm requires a batch of data, batch-based algorithms require significantly more memory than online algorithms.

A. Batch Learning

1) *Support Vector Machine (SVM):* The SVMs are widely known for achieving accurate classification of high-dimensional data. They are also shown recently to perform well in the arena of classifying malicious URLs [13], [14]. For a tutorial on SVMs, we refer the reader to [5]. In this work, we investigate the performance of batch-based SVMs.

B. Online Learning

The online algorithms discussed below operate in rounds. In round t , an online algorithm receives $\mathbf{x}_t$ and predicts $\mathbf{x}_t$'s label as $\hat{y}_t$ using the current model; it then receives the true label, y_t , and updates its model based on $(\mathbf{x}_t, y_t)$.

1) *Online Perceptron (OP):* OP updates $\mathbf{w}$ continuously on error. In particular, $\mathbf{w}$ is updated if the predicted label, $\hat{y}_t = \text{sign}(\mathbf{w}_t \cdot \mathbf{x}_t)$, disagrees with the true label, y_t , of $\mathbf{x}_t$. The update is as follows:

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_t \mathbf{x}_t. \quad (2)$$

OP suffers from a significant drawback: the update rate is fixed and does not take into account the magnitude of classification error, which leads to poor classification accuracy.

2) *Confidence Weighted (CW):* CW is a linear binary classification algorithm recently introduced by Dredze *et al.* [10]. CW captures the notion of confidence in the weight of a feature. Intuitively, the certainty of a feature weight estimate grows as the feature is observed more and more; therefore, the variance for that feature gets lower, *i.e.*, the confidence in the weight of that feature increases. With this confidence notion, CW addresses the drawback of OP through two mechanisms: (i) CW updates the weights of the more confident features less aggressively; and (ii) CW does not change the weights too much but just enough to correct for a mistake.

Formally, CW maintains a Gaussian distribution over the weights with mean μ and covariance matrix Σ . The value μ_i represents what is known about the weight w_i , and the value

$\Sigma_{i,i}$ captures the confidence in the weight of feature i . To classify a new data $\mathbf{x}$, the weight $\mathbf{w}$ is drawn from $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$. In practice, one can pick $\mathbf{w} = \boldsymbol{\mu}$, the average weight vector. The prediction is then as usual: $h(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x})$. The update rule is as follows:

$$(\boldsymbol{\mu}_{t+1}, \Sigma_{t+1}) = \arg \min_{\boldsymbol{\mu}, \Sigma} D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \Sigma) || \mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)), \quad (3)$$

$$\text{s.t. } \Pr_{\mathbf{w} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)} [y_t(\mathbf{w} \cdot \mathbf{x}_t)] \geq \eta. \quad (4)$$

The covariance matrix Σ can be either full or diagonal. η is a configurable parameter and must be larger than 50%. We refer the reader to [8] for more details. The computational complexity of the update is linear in the number of non-zero features in $\mathbf{x}_t$. The memory required is constant in the input data, *i.e.*, the memory for the current $\mathbf{x}$.

3) *Adaptive Regularization of Weights (AROW)*: The final algorithm in this category that we examine is the AROW algorithm by Crammer *et al.* [9]. AROW can be considered as a modification of CW so that the classifier is more robust in the presence of label noise. For example, if ‘whitehouse.gov’ is wrongly labeled as malicious (by an adversary) and fed to CW, then CW will make changes to all features that this URL has so that in the next time slot, if it sees this URL again, it will be likely to flag this URL as malicious. CW, therefore, may drastically increase the weight of the feature “top level domain is .gov”. AROW avoids this drastic behavior by softening the formulation of CW.

Formally, Crammer *et al.* [9] recast the constraint (4) of CW as regularizers. The update rule is now as follows:

$$(\boldsymbol{\mu}_{t+1}, \Sigma_{t+1}) = \arg \min_{\boldsymbol{\mu}, \Sigma} D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \Sigma) || \mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)) + \lambda_1 l_{h^2}(y_t, \boldsymbol{\mu} \cdot \mathbf{x}_t) + \lambda_2 \mathbf{x}_t^T \Sigma \mathbf{x}_t, \quad (5)$$

where $l_{h^2}(y_t, \boldsymbol{\mu} \cdot \mathbf{x}_t) = (\max\{0, 1 - y_y(\boldsymbol{\mu} \cdot \mathbf{x}_t)\})^2$ is the squared-hinge loss suffered using $\boldsymbol{\mu}$ to predict the label for $\mathbf{x}_t$ when its true label is y_t , and λ_1 and λ_2 are configurable parameters.

Similar to CW, the running time of the update is linear in the number of non-zero features in $\mathbf{x}_t$. The memory requirement is constant in terms of the input data. We refer the reader to [9] for more details. To the best of our knowledge, this is the first time that AROW is used in the phishing context.

V. EVALUATION RESULTS

We conducted four sets of experiments in order to: (i) compare batch-based to online algorithms when using just lexical features, (ii) compare lexical to full features, (iii) evaluate the effectiveness of obfuscation-resistant lexical features, and (iv) evaluate the resilience of AROW when working with noisy data. Because of the limited space, we omit the first set of experiments and refer the reader to our technical report [12] for details.²

A. Lexical Features vs. Full Features

We examine the performance of the OP, CW, and AROW algorithms on all pairs of datasets. We implement OP, CW,

²The main result from the first set is that AROW outperforms SVMs when using lexical features. This superior performance and the light-weight properties (light memory requirement and light computation overhead) make AROW, in particular, and online algorithms, in general, better candidates for the URL classification task.

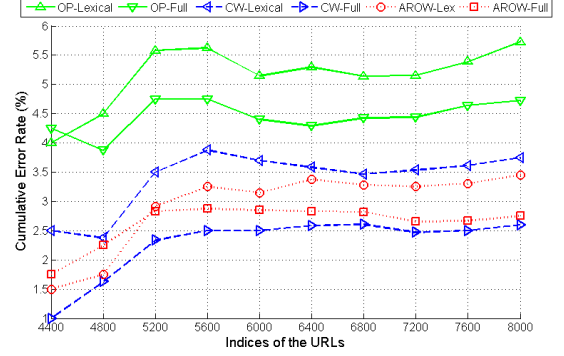


Fig. 1. Performance of OP, CW, and AROW when using Lexical Features versus when using Full Features on Yahoo-Phish

and AROW using Matlab based on the closed form update rules in [8], [9]. Fig. 1 plots the cumulative error rates of the algorithms over time for the Yahoo-Phish pair. We omit the plots for the other pairs due to lack of space; instead, in Table II, we report the cumulative error rates after the last URL for all pairs. Table II also reports the configured parameter η of each experiment involving CW and λ ($=\lambda_1=\lambda_2$) for each experiment involving AROW. Both λ and η are configured using cross validation. Based on the plot and the table, we make the following observations.

Consider CW and AROW, the cumulative error rates for the pair All Good - All Bad are always larger than all other pairs regardless of using lexical or full features. This suggests that one should build two separate classifiers for PhishTank and MalwarePatrol instead of building a single classifier for both. This agrees with the discussion in Section III: these are datasets with different characteristics due to their different purposes. Therefore, we subsequently focus our discussion on the other four pairs. For these pairs, CW and AROW outperform OP regardless of using lexical or full features; moreover, CW and AROW have comparable performance when using lexical features: AROW slightly edges CW on Yahoo-Phish while CW slightly edges AROW on the other. Finally, the gain of using full features over lexical features is only about 1% for both CW and AROW across all the pairs of interest. Using lexical features alone, AROW achieves 96–97% of accuracy while CW achieves 96–98% of accuracy.

Summary. This set of experiments shows that using lexical features alone leads to comparable classification accuracy to full features (only 1% difference). The high accuracy and the lightweight properties of lexical features make a strong case for using lexical features alone.

B. Obfuscation-Resistant (OR) Lexical Features

Here we evaluate the effectiveness of the obfuscation-resistant (OR) lexical features when using AROW. Table III reports the performance of AROW when the OR features are not used (only auto-selected features are used) and when the OR features are used. From the results, we can see that the OR features boost the classification accuracy across all pairs of datasets. The reduction of the cumulative error rate ranges from 9% (on Yahoo-Phish) up to 27% (on DMOZ-Malware.)

To better understand the improvement, we look at the changes of both the number of mis-classified malicious URLs,

TABLE II
CUMULATIVE ERROR RATE OF THE OP, CW, AND AROW ALGORITHMS ON ALL DATASETS AFTER THE LAST URL

Dataset	Cumulative Error Rate of OP (%)			Cumulative Error Rate of CW (%)			Cumulative Error Rate of AROW (%)		
	Lexical Ftrs	Full Ftrs	Gain	Lexical Ftrs	Full Ftrs	Gain	Lexical Ftrs	Full Ftrs	Gain
Yahoo-Phish	5.72	4.73	0.99	3.75 ($\eta = 73\%$)	2.60 ($\eta = 84\%$)	1.15	3.45 ($\lambda = 0.5$)	2.75 ($\lambda = 0.5$)	0.70
Yahoo-Malware	3.27	2.40	0.87	2.17 ($\eta = 66\%$)	1.62 ($\eta = 66\%$)	0.55	3.05 ($\lambda = 5.0$)	1.98 ($\lambda = 5.0$)	1.07
DMOZ-Phish	6.90	4.73	2.17	3.52 ($\eta = 86\%$)	2.52 ($\eta = 79\%$)	1.00	3.60 ($\lambda = 50$)	2.77 ($\lambda = 50$)	0.83
DMOZ-Malware	3.75	3.20	0.55	3.23 ($\eta = 58\%$)	2.35 ($\eta = 62\%$)	0.87	3.75 ($\lambda = 0.5$)	2.75 ($\lambda = 5.0$)	1.00
All Good - All Bad	5.43	4.31	1.12	4.07 ($\eta = 58\%$)	3.14 ($\eta = 58\%$)	0.87	5.48 ($\lambda = 5.0$)	4.00 ($\lambda = 5.0$)	1.48

TABLE III
PERFORMANCE OF AROW WHEN NOT USING AND WHEN USING OBFUSCATION-RESISTANT (OR) LEXICAL FEATURES AFTER THE LAST URL

Dataset	Cumulative Error Rate (%)			# Mis-Classified Benign URLs (FP)			# Mis-Classified Malicious URLs (FN)		
	w/o OR Ftrs	with OR Ftrs	Gain (Gain Pctg)	w/o OR Ftrs	with OR Ftrs	Gain	w/o OR Ftrs	with OR Ftrs	Gain
Yahoo-Phish	3.92 ($\lambda = 0.5$)	3.45 ($\lambda = 0.5$)	0.37 (9%)	47	55	-8	110	83	27
Yahoo-Malware	3.70 ($\lambda = 5.0$)	3.05 ($\lambda = 5.0$)	0.65 (18%)	88	77	11	60	45	15
DMOZ-Phish	4.05 ($\lambda = 5.0$)	3.60 ($\lambda = 50$)	0.45 (11%)	50	53	-3	112	91	21
DMOZ-Malware	5.12 ($\lambda = 0.5$)	3.75 ($\lambda = 0.5$)	1.37 (27%)	22	41	-19	183	109	74

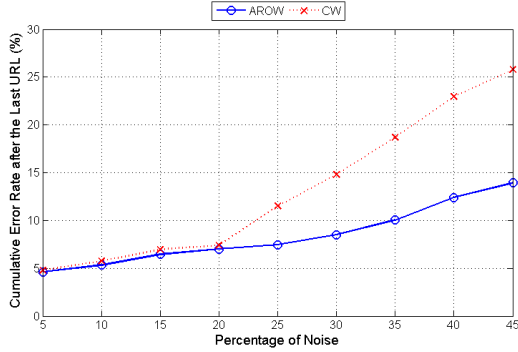


Fig. 2. Performance of AROW and CW on Yahoo-Phish with Various Amount of Noise

i.e., false negatives (FNs), and the number of mis-classified benign URLs, *i.e.*, false positives (FPs). From Table III, we can see that the improvement mainly comes from the reduction of the number of mis-classified malicious URLs. In particular, we reduce the number of FNs (ranging from 15 to 74) for a modest increase in the number of FPs (ranging from 3 to 19.)

Summary. These experiments show that the obfuscation-resistant lexical features effectively improve the overall classification accuracy by catching more phishing URLs.

C. The Resilience of AROW to Noisy Data

In this last set of experiments, we examine how resilient AROW is to noisy data. Here we report the results on Yahoo-Phish (the other pairs give similar results.) To create noise, we randomly select a number of URLs and change their labels from malicious to benign or vice versa. Fig. 2 shows the cumulative error after the last URL of both AROW and CW on Yahoo-Phish with various amount of noise. We make the following observations: First, AROW consistently achieves better classification accuracy than CW; moreover, the noisier the dataset, the larger the difference between the performance of AROW and CW. Second, AROW is able to maintain very high accuracy (about 95%) when there is a modest amount of noise (from 5 to 10%) and high accuracy (above 90%) even when there is a moderate amount of noise (from 10 to 30%).

Summary. AROW can achieve high classification accuracy, higher than CW, when working with noisy data.

D. Understanding the Performance

In addition to the above sets of experiments, we also analyzed the characteristics of phishing URLs to better understand

(i) why online algorithms outperform batch-based algorithms in classifying phishing URLs, and (ii) why the advanced online algorithms, namely, AROW and CW, outperform the classical OP algorithm in this context. The key answer coming out of our analysis is that maintaining long term memory and updating models quickly are essential to achieve better classification accuracy. Due to lack of space, we omit the details and refer the reader to the technical report [12].

VI. CONCLUSION

In this work, we propose PhishDef, a system that performs proactive, on-the-fly classification of phishing URLs using only lexical features and the AROW algorithm. By using only lexical features, PhishDef reduces the page loading latency and avoids reliance on remote servers. By implementing the AROW algorithm, PhishDef achieves high classification accuracy, even with noisy data, while at the same time having low computation and memory requirements.

REFERENCES

- [1] Google Safe Browsing API. <http://code.google.com/apis/safebrowsing/>.
- [2] MalwarePatrol. <http://www.malwarepatrol.net>.
- [3] Microsoft Smart Screen. <http://windows.microsoft.com/en-US/windows-vista/SmartScreen-Filter-frequently-asked-questions>.
- [4] PhishTank. <http://www.phishtank.com>.
- [5] SVM Tutorials. <http://www.svms.org/tutorials/>.
- [6] Yahoo URL Random Generator. <http://random.yahoo.com/bin/ryl>.
- [7] Anti Phishing Working Group. Q4 2009 Report. <http://antiphishing.org>.
- [8] K. Crammer, M. Dredze, and F. Pereira. Exact Convex Confidence-Weighted Learning. In *Proc. of NIPS '08*.
- [9] K. Crammer, A. Kulesza, and M. Dredze. Adaptive Regularization Of Weight Vectors. In *Proc. of NIPS '09*.
- [10] M. Dredze, K. Crammer, and F. Pereira. Confidence-Weighted Linear Classification. In *Proc. of ICML '08*.
- [11] S. Garera, N. Provos, M. Chew, and A. D. Rubin. A Framework for Detection and Measurement of Phishing Attacks. In *WORM '07*.
- [12] A. Le, A. Markopoulou, and M. Faloutsos. Technical Report: "PhishDef: URL Names Say It All". <http://www.ics.uci.edu/~anhml/publications.html>. Also on *arxiv:1009.2275*, Sep 2010.
- [13] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker. Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs. In *Proc. of SIGKDD '09*.
- [14] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker. Identifying Suspicious URLs: An Application of Large-Scale Online Learning. In *Proc. of ICML '09*.
- [15] T. McCall. Gartner survey shows phishing attacks escalated in 2007. <http://www.gartner.com/it/page.jsp?id=565125>.
- [16] P. Prakash, M. Kumar, R. R. Kompella, and M. Gupta. PhishNet: Predictive Blacklisting to Detect Phishing Attacks.
- [17] Team Cymru. IP to ASN Mapping. <http://www.team-cymru.org/Services/ip-to-asn.html>.
- [18] C. Whittaker, B. Ryner, and M. Nazif. Large-Scale Automatic Classification of Phishing Pages. In *Proc. of NDSS '10*.